CryptoChain Analyzer Dashboard

Final Report · Cryptography and Cybersecurity (UAX, 2025-26) · Prof. Jorge Calvo

Author: **Alejandro Déniz Solana** (@aledeniiz) · Repo:
github.com/aledeniiz/blockchain-dashboard-aledeniiz

1. Cryptographic metrics displayed

The dashboard pulls live data from the public mempool.space REST API (with a Blockstream fallback) and exposes six cryptographic metrics directly tied to the theory of Topic 7. Every metric is recomputed from primitives using Python's `hashlib` and the standard library — no third-party crypto package is trusted for the verification step.

Module	Metric	Theoretical link
M1	Difficulty, leading-zero bits, hash rate	Proof of Work; difficulty = genesis_target / target
M1	Inter-block time histogram	Exp(λ = 1/600 s) — mining is a memoryless Poisson process
M2	80-byte header parse + manual SHA256² check	PoW validity: SHA256(SHA256(header)) < target
M2	Compact 'bits' decoding into 256-bit target	T = coefficient × 256^(exponent – 3)
M3	Difficulty per epoch, ratio actual/600 s	Retarget every 2 016 blocks
M5	Merkle path verification (optional)	Inclusion proof in $O(\log_2 n)$ hashes
M6	51 % attack cost, Nakamoto §11 attack probability (optional)	Random-walk gambler's-ruin model of catch-up
M7	Difficulty predictor (optional, second AI)	Supervised regression on $\log_{10}$(difficulty) per epoch

The header verification (M2) is the most important sanity check: for any block the dashboard fetches its raw 80-byte header, parses each little-endian field with `struct`, computes `SHA256(SHA256(header))`, reverses the byte order, decodes the target from the 'bits' field, and confirms that the resulting 256-bit integer is strictly below the target. The same block hash printed on any explorer is reproduced byte-for-byte from first principles — closing the loop between theory and live data.

2. AI component (M4) — Isolation Forest

Why this model. Bitcoin inter-block times are a memoryless Poisson arrival process, so the inter-arrival distribution is Exponential(λ = 1/600 s). Detecting blocks that deviate from this distribution is an unsupervised problem: there are no ground-truth anomaly labels available. Isolation Forest is a natural fit because (i) it does not require labels, (ii) it scales linearly in the number of samples, and (iii) it isolates outliers in fewer random splits than inliers, which matches the geometric intuition of an exponential tail.

Features. `log_inter = log(inter_block_seconds + 1)` stabilises the heavy tail of the exponential, and `tx_count` contributes a secondary signal (unusually empty or full blocks may co-occur with timing anomalies). Features are standardised with `StandardScaler` before being fed to `IsolationForest(n_estimators=100, contamination=c, random_state=42)`, where *c* is exposed as a slider to the user.

Evaluation. The unsupervised setting precludes accuracy/F1, so the dashboard reports two complementary diagnostics. (1) The empirical histogram of inter-block times is overlaid with the theoretical Exp(λ = 1/600 s) PDF: the body of the distribution should match and the tail should hold the flagged

anomalies. (2) The fraction of blocks flagged at the chosen contamination rate is reported alongside a scatter plot of inter-block time vs. wall-clock time, so the user can visually confirm the model is selecting the longest gaps. On a 50-block window pulled at the time of writing, the model flagged the expected number of tail events; on a 200-block window the signal-to-noise ratio improves further. **Limitations.** With a small block window the contamination parameter is non-trivial to choose; with a large window the assumption of a stationary λ becomes weaker because the 2 016-block retarget changes the rate. Both are documented in the module's expander.

3. Optional modules (M5, M6, M7)

M5 — Merkle Proof Verifier. For any block the user picks, the module fetches every txid via `/block/<hash>/txids`, converts the displayed hex strings to internal little-endian byte order, walks the tree bottom-up duplicating the last hash on odd levels (the well-known CVE-2012-2459 quirk), and records the sibling hashes along the path of the chosen transaction. The recomputed root is then compared against the header's `merkle_root`, and the path itself is re-verified independently in the SPV style — confirming that only $log_2(n)$ hashes (32 B each) are needed to prove inclusion.

M6 — Security Score. Two analyses are exposed: (i) the operating cost per hour to match the network hash rate, computed as `H · J/TH · $/kWh` with user-configurable ASIC efficiency and electricity price, and (ii) the probability that an attacker holding fraction q of the hash power overtakes the honest chain after z confirmations, computed with Nakamoto's §11 formula:

$$P = 1 - \Sigma_{k=0..z} \left(\lambda^k \cdot e^{-\lambda} / k!\right) \cdot \left(1 - (q/p)^{z-k}\right), \quad \lambda = z \cdot q/p$$

The implementation was validated against the table in the original Bitcoin whitepaper: the canonical reference points (q = 0.10 z = 5; q = 0.30 z = 5; q = 0.30 z = 10) match to seven decimals, and a high-precision `mpmath` reference at 50 decimal places agrees to better than 1e-9. The result is plotted on a logarithmic axis so the exponential decay against z is visible immediately — and confirms why the canonical six-confirmation rule is considered safe for $q \le 0.10$.

M7 — Difficulty Predictor (second AI approach). A supervised regression model complements M4's unsupervised Isolation Forest with a different problem framing: predict the value of the next 2 016-block difficulty adjustment. Because Bitcoin difficulty has grown roughly exponentially with hashrate, we model $\log_{10}(\text{difficulty})$ rather than raw difficulty; in that space a linear model is the natural baseline. Features at epoch t include the monotonic *epoch_idx* (secular trend), the current log-difficulty, the ratio *actual_avg_time / 600* (the variable that drives the retarget rule), and a one-step log-momentum term. We compare three models — Linear regression, Ridge ($\alpha = 1$), and a 100-tree Random Forest — on a *chronological* 75/25 split (random shuffles would leak future information). Evaluation reports MAE in the model's working space, MAPE in raw difficulty (the operational error), and R^2 on the test split. The Linear and Ridge models track the next-epoch difficulty within roughly ±1 % MAPE on recent epochs.

Together (M4 + M7). The two AI components cover the two main flavours of ML applied to blockchain data: unsupervised tree-ensemble anomaly detection on a memoryless arrival process (M4), and supervised linear / ensemble regression for a time-series forecast (M7). They are evaluated with appropriate, distinct metrics rather than a single ill-fitting one — which is the substantive point of the rubric's C2 criterion.

4. Engineering notes

The dashboard is built with Streamlit, plots with Plotly, and refreshes automatically every 60 s through `st.cache_data(ttl=60) + a time.sleep + st.rerun()` loop. The API client uses a primary/fallback pattern to survive transient outages of either provider. The cryptographic helpers — `bits_to_target`, `bits_to_difficulty`, `sha256d` — are reused by every module, making it straightforward to add further analyses without touching the network layer.

Automated tests. A 25-case `pytest` suite under `tests/` protects every cryptographic primitive: `bits→target` decoding, double-SHA-256, the full Merkle tree (including the CVE-2012-2459

odd-duplication quirk and sibling tampering), the Nakamoto §11 formula against six whitepaper reference points plus monotonicity and edge cases, and four *live* end-to-end checks that fetch the latest mainnet block and reproduce its hash and Merkle root byte-for-byte. The live tests are skipped automatically when the API is unreachable. All 25 pass.

5. References

[1] Nakamoto, S. (2008). *Bitcoin: A Peer-to-Peer Electronic Cash System.* §6 (Difficulty), §7 (Merkle Trees), §11 (Calculations). https://bitcoin.org/bitcoin.pdf

[2] mempool.space REST API documentation. https://mempool.space/docs/api/rest

[3] Blockstream Esplora HTTP API reference. https://github.com/Blockstream/esplora/blob/master/API.md

[4] Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). *Isolation Forest.* ICDM 2008. https://doi.org/10.1109/ICDM.2008.17

[5] scikit-learn user guide — Outlier detection / Isolation Forest. https://scikit-learn.org/stable/modules/outlier_detection.html

[6] scikit-learn user guide — Linear models (Linear / Ridge / Random Forest regression). https://scikit-learn.org/stable/modules/linear_model.html

[7] Bitcoin Core developers — block header serialization. https://developer.bitcoin.org/reference/block_chain.html

All code in this report can be regenerated from the repository: every metric, every chart, every claim is backed by an inspection step exposed in the dashboard UI. The PDF itself is built by `report/build_report.py`, so the report is reproducible from source.